NXT WAVE™

WELCOME

NXT WAVE™

State, Behaviour & Constructor

Recap (Part 1/2)

Introduction to OOP

- Object-Oriented Programming
- The Problem Without OOP
- Class
    - The Blueprint



NXT WAVE™

Recap (Part 2/2)

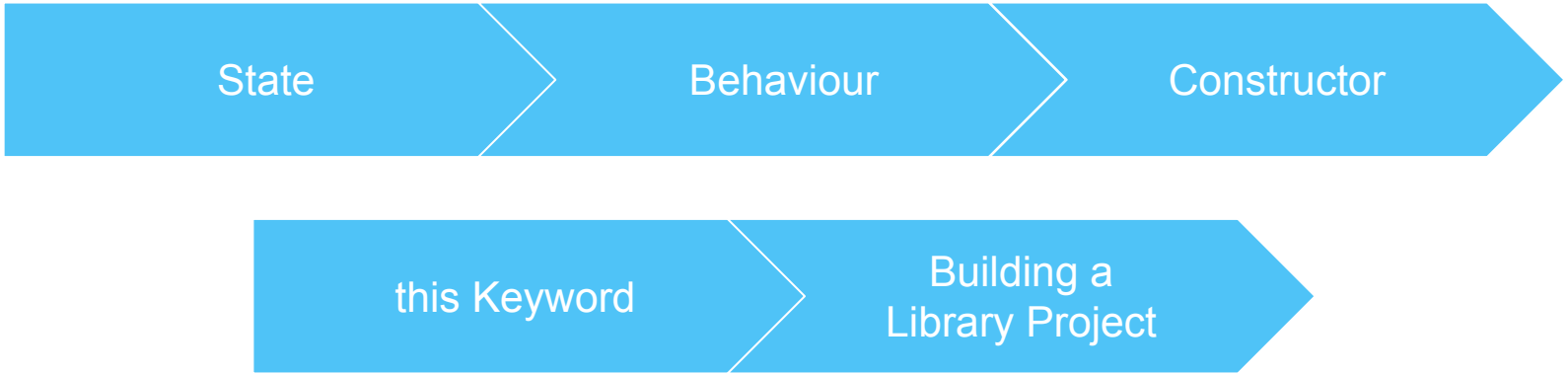
EER Modeling Concepts

- Object - The Real Thing
    - Creating Multiple Objects from One Class
- Real-world Applications of OOP

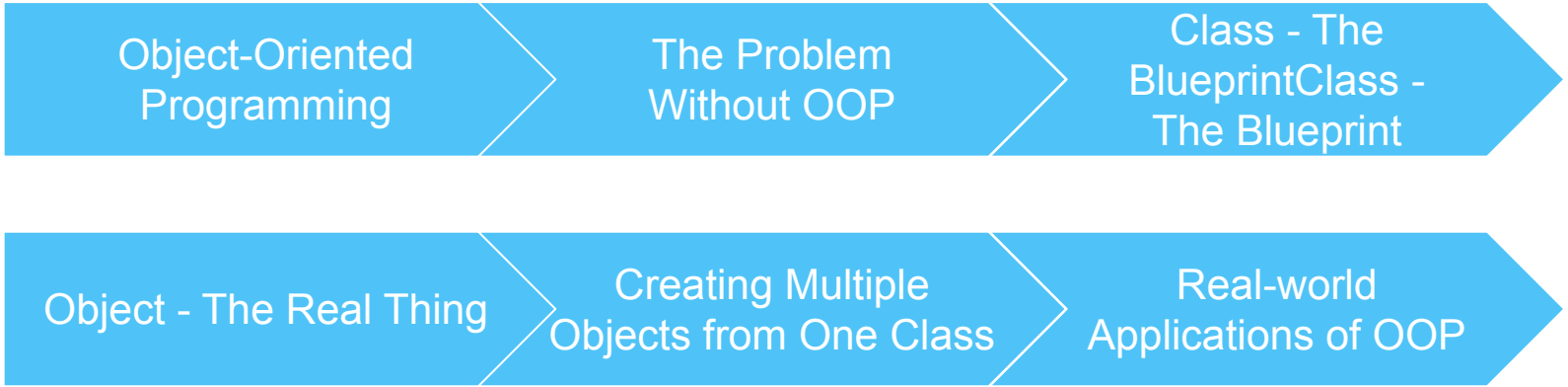


Agenda for Today's Session

State, Behaviour & Constructor



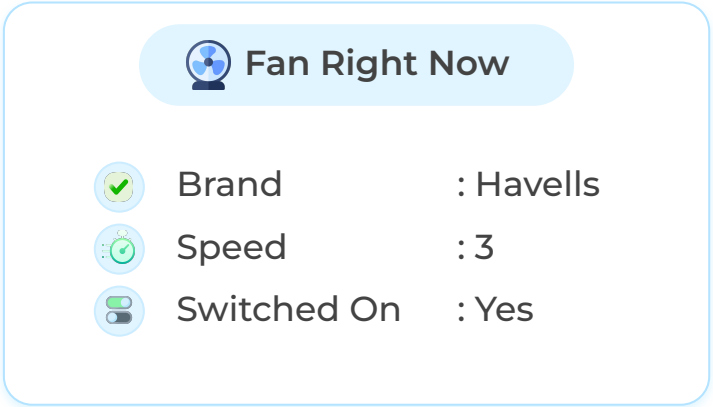
State, Behaviour & Constructor



Real-world Example

Think about a Fan in a room.

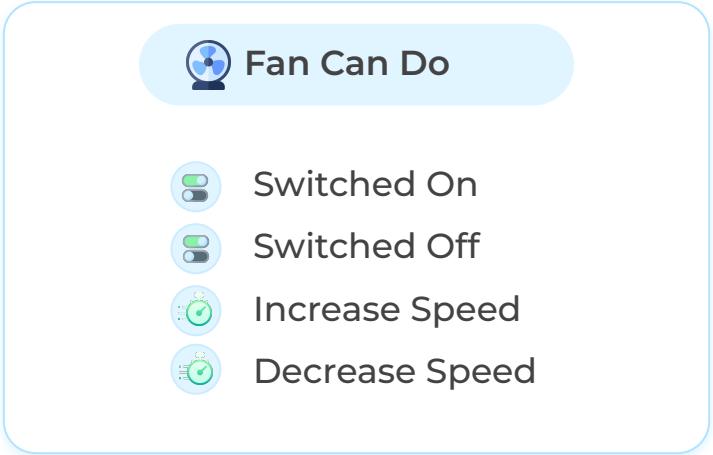
At any moment, the fan has some current information:



This is the fan's current state

Real-world Example

And the fan has some actions it can do:



Now here is the important part

when we perform an action, the **information changes**

Real-world Example



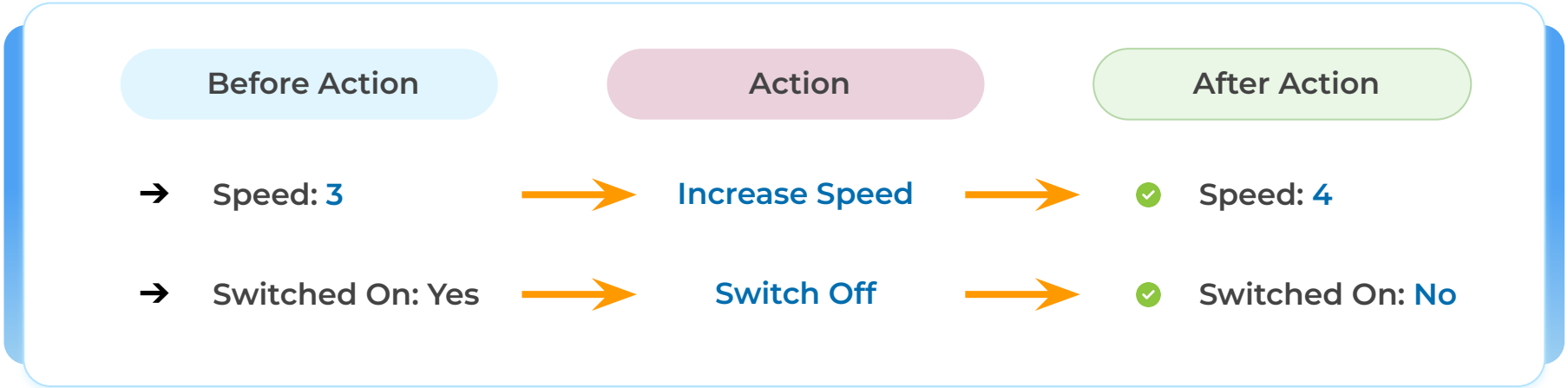
And the fan has some actions it can do:

Fan Can Do

- Switched On
- Switched Off
- Increase Speed
- Decrease Speed

when we perform an action, the **information changes**

Real-world Example



The action changed the fan's **current information**, this is how state and **behaviour** are **connected**

01

State



NXT WAVE™

State

- State is the current information an **object holds right now**
- It can change over time as **actions happen**



Real-world Examples of State

Book

State:

- Title : “Harry Potter”
- Author : “Rowling”
- Available : true

Phone

State:

- Brand : “Samsung”
- Battery : 80%
- Available : On

Bank

State:

- Holder : “Ali”
- Balance : 50000



In OOP, the state is stored in attributes (also called data or fields)

Pseudocode

```
CLASS Book:
    ATTRIBUTES:
        title
        author
        available
END CLASS
```

This is the STATE



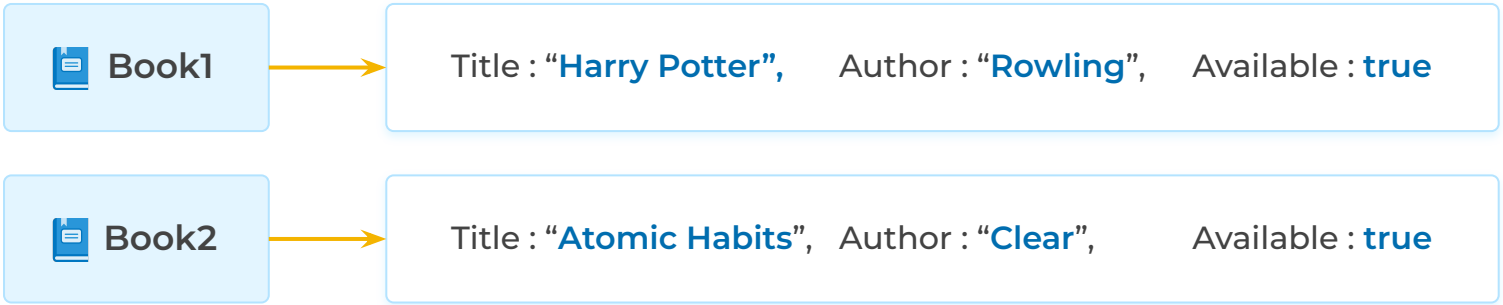
Ea

boo

boo

Pseudocode

- Each object created from this class will have its own state:



The state of book1 is different from the state of book2. Each object is **independent**

02

Behaviour



Behavior

Behaviour is what an object can do — the actions it can perform

When a behaviour runs, it usually reads or changes the object's state

Real-world Examples of Behaviour

Book	Phone	Bank Account
Behaviour:	**Behaviour**:	**Behaviour**:
borrow()	makecall()	deposit()
returnBook()	takePhoto()	withdraw()
display()	charegeBattery()	checkBalance()

In OOP, behaviour is written as methods (functions that live inside a class)

State, Behaviour & Constructor

In Our Code (Pseudocode)

```
CLASS Book:
    ATTRIBUTES:
        title
        author
        available

    METHODS:
BEHAVIOUR
        borrow()
        returnBook()
        display()
END CLASS
```

This is the Behaviour

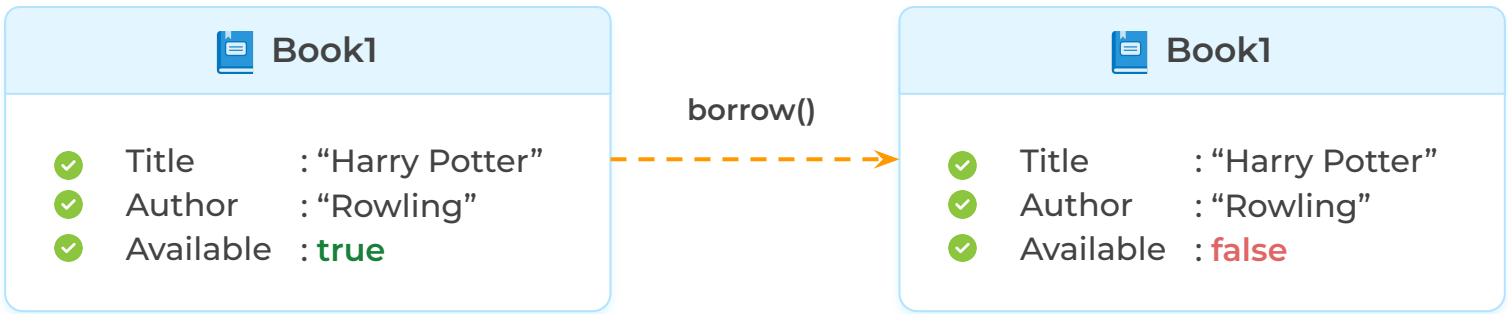


NXT WAVE™

State Changes As Behaviour Happens

This is the key connection between **state and behaviour**. Let us see it visually:

Example: Borrowing a Book



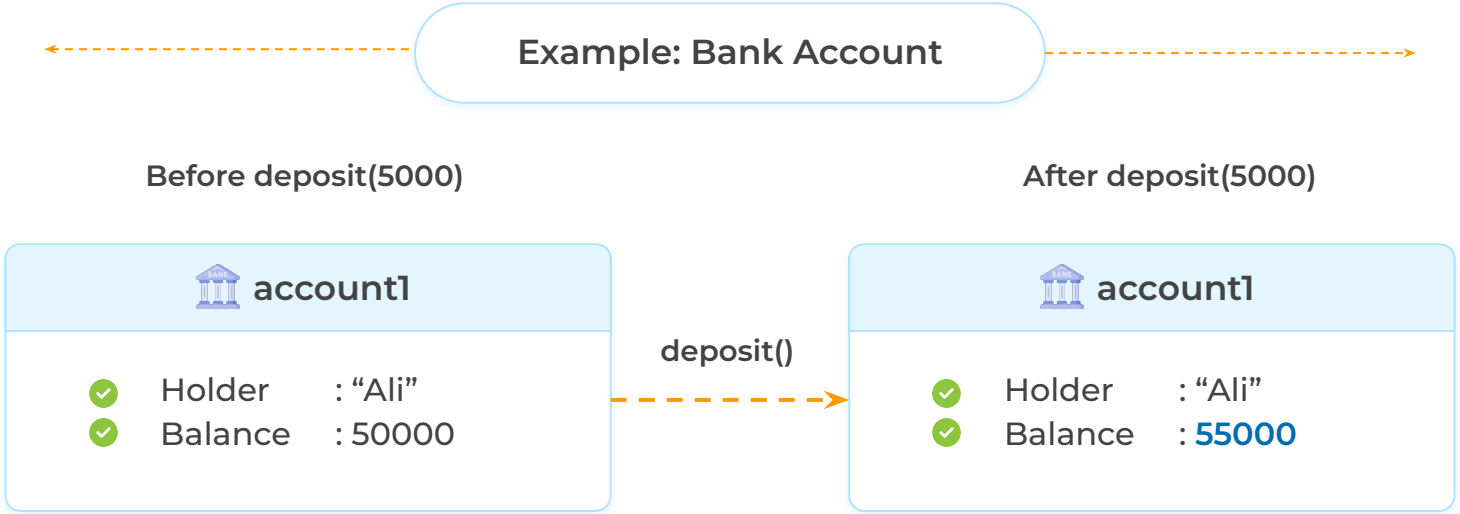
State Changes As Behaviour Happens



The borrow() method changed the **state of book1**

Only "available" changed. The rest **stayed the same**

State Changes As Behaviour Happens



State, Behaviour & Constructor

The Rule

	State (Attributes)	Behaviour (Methods)
What it is	The data an object holds right now	The actions an object can perform
Does it change?	Yes — it changes when methods run	No — the methods stay the same
Example	available: true → available: false	borrow() always does the same job

State, Behaviour & Constructor

Setting Up the Starting State



- When we create a new book, we need to give it a **title**, **author**, and **set available** to **true**
- In OOP, there is a special method that **runs automatically** when an object is **created**. It sets up the **starting state**
- This special method is called a **Constructor**

03

Constructor



NXT WAVE™

State, Behaviour & Constructor

Constructor



A Constructor is a **special method** that:

- **Runs automatically** when we create a new object
- Sets up the object's starting state (initial values for attributes)
- We never call it **manually** — it **runs** on its own at the time of **creation**

How the Constructor Works

We write this

```
book1 = NEW Book("Harry Potter", "Rowling", true)
```

- NEW triggers the constructor automatically
- title
- author
- available



How the Constructor Works

Inside the class, the constructor receives these values:

```
CONSTRUCTOR(title, author, available):
    set this.title     = title       // "Harry Potter"
    set this.author    = author       // "Rowling"
    set this.available = available    // true
```



- This runs automatically. We do not call it **ourselves**

How the Constructor Works



Result - book1 is now ready to use:

Book1

- Title : “Harry Potter”
- Author : “Rowling”
- Available : **true**

State, Behaviour & Constructor

Writing-Constructor (Pseudocode)

```
CLASS Book:
    ATTRIBUTES:
        title
        author
        available

    CONSTRUCTOR(title, author):
this.title = title
        this.author = author
        this.available = true
    METHODS:
        borrow()
        returnBook()
        display()
END CLASS
```

We do not pass available from outside. Every new book always starts as available, so the constructor sets it to true automatically



NXT WAVE™

Quiz Time!



04 this Keyword



NXT WAVE™

This

In the constructor, we wrote `this.title = title`. What does this mean?

The Problem Without `this` Look at the constructor again:

```
CONSTRUCTOR(title, author):
    title = title
    author = author
```

Confusing! Which "title" is which?

Same problem!



This

We have two things with the same name:

- The parameter `title` (the value passed in from outside)
- The attribute `title` (the data that belongs to the object)



How does the code know which one we mean?

The Solution - this

`this` means "the object I belong to"

```
CONSTRUCTOR(title, author):
    this.title = title
```

The parameter (value coming from outside)

The attribute of THIS object (the one being created)



State, Behaviour & Constructor

This

`this` → book1 (the object being created right now

```
book1 = NEW Book("Harry Potter", "Rowling")
```



Inside the constructor:

```
this.title = title
```

book1's title = "Harry potter" (parameter)

```
this.author = author
```

book1's title = "Rowling" (parameter)

State, Behaviour & Constructor

This

```
book1 = NEW Book("Harry Potter", "Rowling") Inside the constructor:
```

`this` → book1 (the object being created right now

```
this.title = title
```

book1's title	=	"Harry potter" (parameter)

```
this.author = author
```

book1's title	=	"Rowling" (parameter)



NXT WAVE™

State, Behaviour & Constructor

This

```
book2 = NEW Book("Atomic Habits", "Clear")
```

Inside the constructor:

`this` → book2 (the object being created right now

```
this.title = “Atomic Habits”

this.author = “clear”
```



NXT WAVE™

The Rule

`this` always points to the object that is being used right now

- When we create `book1`, this means `book1`
- When we create `book2`, this means `book2`

`this` Inside Methods Too

`this` is not just for the constructor. We use it inside any method to refer to the current object

Example: The borrow() Method

```
CLASS Book:
    ATTRIBUTES:
        title
        author
        available
```



State, Behaviour & Constructor

The Rule

```
CONSTRUCTOR(title, author):
        this.title = title
        this.author = author
        this.available = true

    METHOD borrow():
        IF this.available == true:
            this.available = false
            PRINT this.title + " borrowed"
        ELSE:
            PRINT this.title + " is not available"
        END IF
    END METHOD
```

Check THIS object's state

Change THIS object's state

Read THIS object's state

NXT WAVE™

The Rule

```
    METHOD returnBook():
        this.available = true
        PRINT this.title + " returned"
    END METHOD

    METHOD display():
        PRINT "Title: " + this.title
        PRINT "Author: " + this.author
        PRINT "Available: " + this.available
    END METHOD
END CLASS
```

Change THIS object's state

Read THIS object's state

Running It Step by Step

```
book1 = NEW Book("Harry Potter", "Rowling")
book2 = NEW Book("Atomic Habits", "Clear")
```

Step 1: Constructor runs for book1

Book1

- Title : “Harry Potter”
- Author : “Rowling”
- Available : **true**

Running It Step by Step

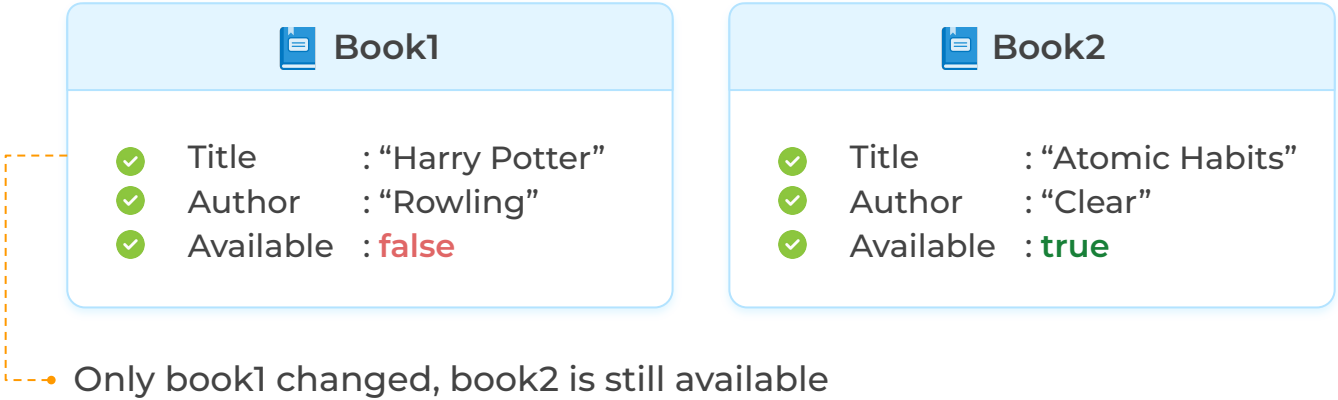
Step 2: Constructor runs for book2

Book1	
Title	: “Harry Potter”
Author	: “Rowling”
Available	: **true**

Book2	
Title	: “Atomic Habits”
Author	: “Clear”
Available	: **true**

Running It Step by Step

Step 3: `book1.borrow()` (here this = book1)



Output

```
“Harry potter borrowed”
```

Running It Step by Step

Step 4: `book2.display()` (here this = book2)

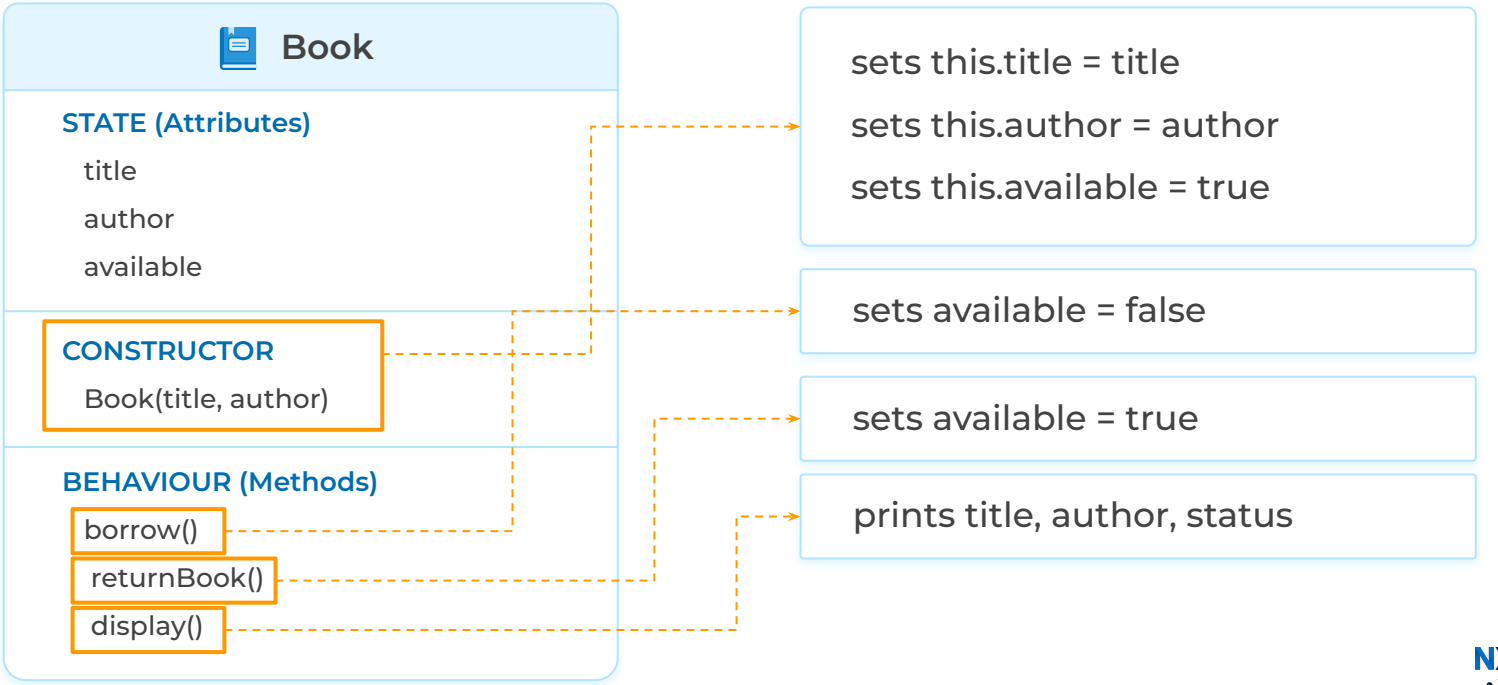
Output

```
Title: Atomic Habits
Author: Clear
Available: true
```



`this` pointed to `book2`, so it read `book2`'s data – not `book1`'s

All Together - The Complete Picture



State, Behaviour & Constructor

All Together - The Complete Picture



- Constructor sets up the **starting** state
- Methods **read** or **change** the state
- "`this`" connects methods to the right object

05

Building a Library Project



Example - Member Class Of Our Library

- Let us create **one more class** to grow our **Library project**
- A library also has Members who **borrow books**

Identifying state and behaviour



All Together - The Complete Picture



LIBRARY MEMBER

State (what we know):

- name
- memberId
- borrowedBooks (starts empty)

Behaviour (what they can do):

- borrowBook(book)
- returnBook(book)
- displayInfo()

Writing-Member Class (Pseudocode)

```
CLASS Member:
    ATTRIBUTES:
        name
        memberId
        borrowedBooks
    CONSTRUCTOR(name, memberId):
        this.name = name
        this.memberId = memberId
        this.borrowedBooks = []

METHOD borrowBook(book):
        ADD book TO this.borrowedBooks
        PRINT this.name + " borrowed " + book
    END METHOD
```

Starts with **no books**



Writing-Member Class (Pseudocode)

```
  METHOD returnBook(book):
        REMOVE book FROM this.borrowedBooks
        PRINT this.name + " returned " + book
    END METHOD

    METHOD displayInfo():
        PRINT "Name: " + this.name
        PRINT "Member ID: " + this.memberId
        PRINT "Books borrowed: " +
this.borrowedBooks.length
    END METHOD
END CLASS
```



State, Behaviour & Constructor

Creating Member Objects

```
member1 = NEW Member("Ali", "M001")
member2 = NEW Member("Sara", "M002")
member1.borrowBook("Harry Potter")
member1.borrowBook("Clean Code")
member1.displayInfo()
```

Output

```
Ali borrowed Harry Potter
Ali borrowed Clean Code
Name: Ali
Member ID: M001
Books borrowed: 2
```

Quiz Time!

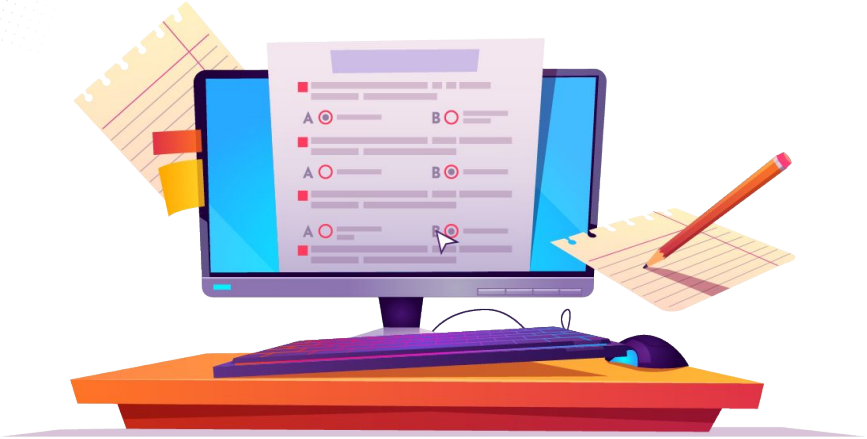


Key Takeaways

- State
- Behaviour
- Constructor
- this Keyword
- Building a Library Project



Practice



Next Session

Encapsulation

NXT WAVE™

THANK YOU

NXT WAVE™

ALL THE BEST